DSCI 560

Lecture 7: Python, C, CUDA

Data Science Practicum

Young Cho
Department of Electrical Engineering
University of Southern California

Some of the Slides borrowed from NVIDIA

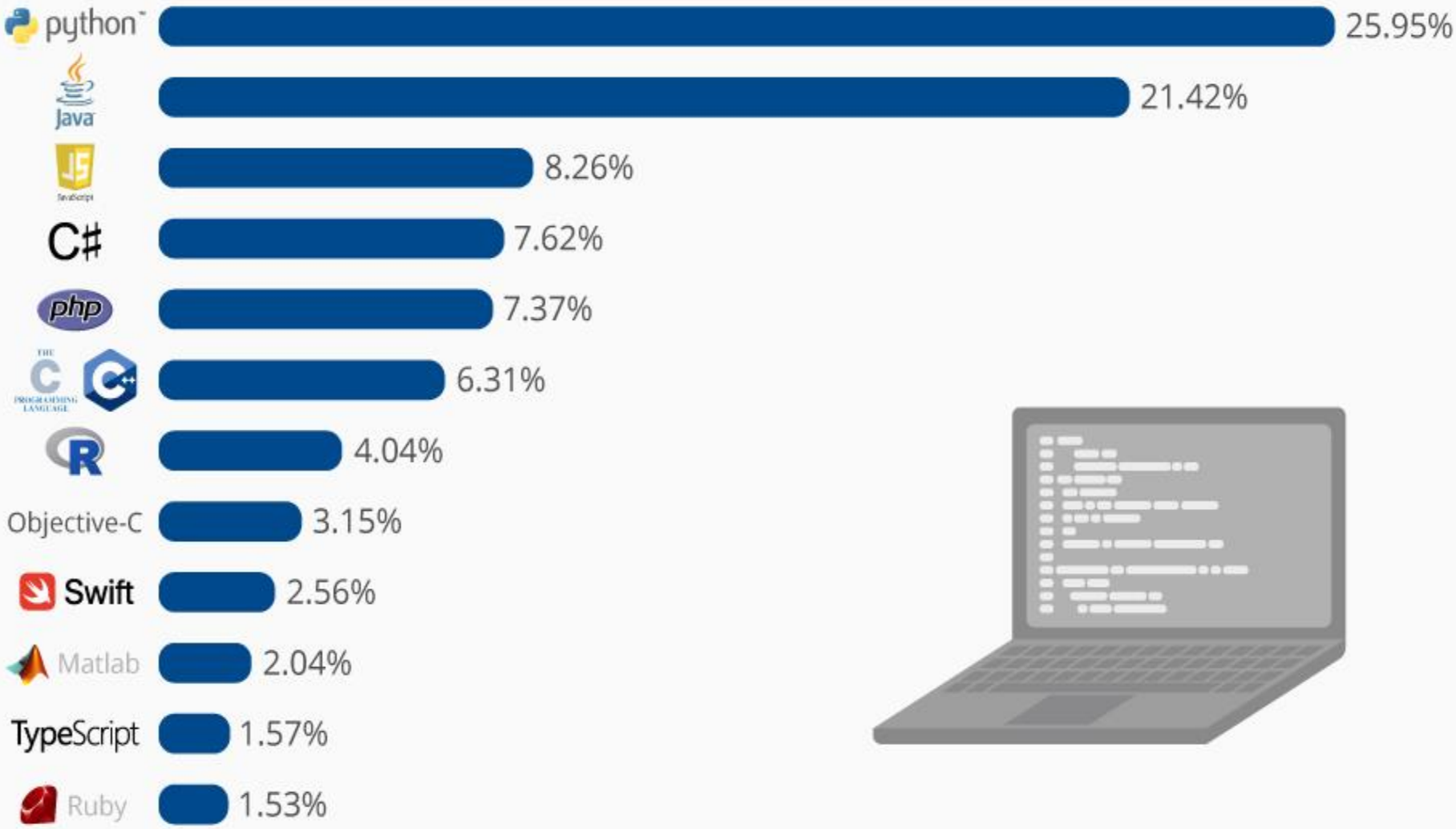
Development Tool: Python

- Started by Guido Van Rossum as a hobby
- Now, the Most Popular Language
  - Also means everyone already knows
- Open Source
- Versatile
- Lots of prebuilt packages
  - This is the KEY
- BUT, only used like a wrapper
  - Need to go beyond!!

Guido Van Rossum

The Most Popular Programming Languages

Share of the most popular programming languages in the world*



* Based on the PYPL-Index, an analysis of Google search trends for programming language tutorials.

@StatistaCharts

Source: PYPL

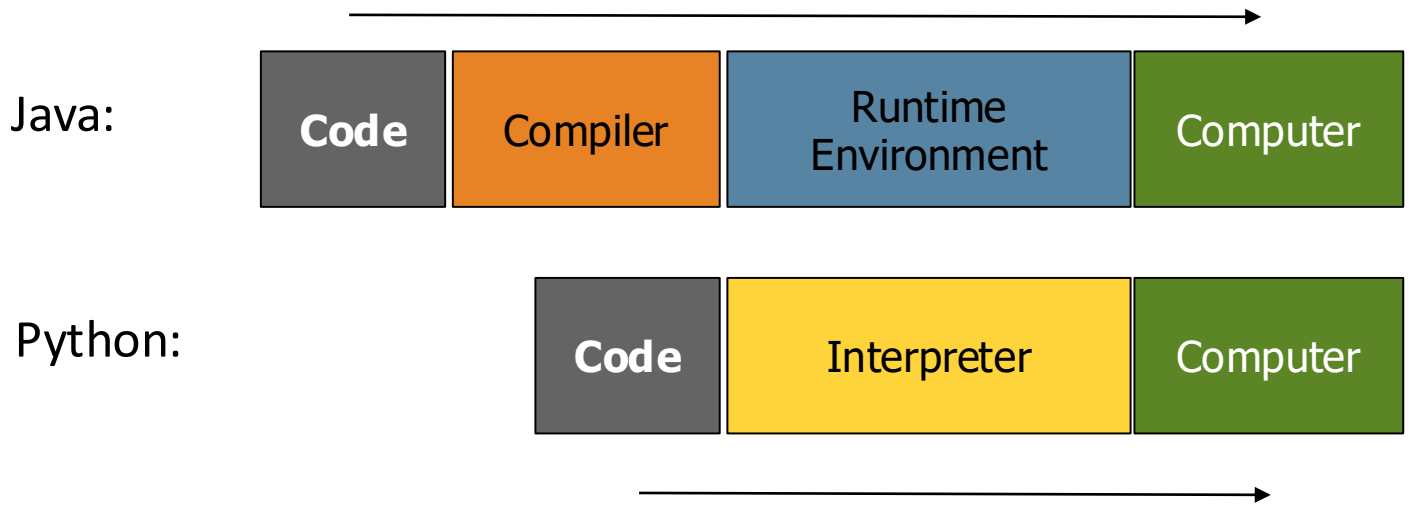
statista

2025 IEEE Ranking based on Types of Developments

Language Rank	Types	Spectrum Ranking
1. Python		100.0
2. C		99.7
3. Java		99.5
4. C++		97.1
5. C#		87.7
6. R		87.7
7. JavaScript		85.6
8. PHP		81.2
9. Go		75.1
10. Swift		73.7

Interpreted Languages

- **Interpreted**
  - Not compiled like Java
  - Code is written and then directly executed by an **interpreter**
  - Type commands into interpreter and see immediate results



Python vs. C

- Python is high-level, easy, but slower.
- C is lower-level, compiled, and much faster.
- C gives control over memory and hardware.
- Many Python libraries (NumPy, TensorFlow) are written in C.

Key Differences

- Compiled vs Interpreted
  - Python: interpreted, dynamic
  - C: compiled, static typing
- Data Types/Memory
  - Python: Automatic (Easy but with overhead)
  - C: Manual (Highest level of customization)
- Syntax differences
  - Indentation vs braces {}

First Program

- Python:

```
print("Hello, World!")
```

- C:

```
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Variables & Types

Python:

- x = 10
- y = 3.14

C:

- int x = 10; // 32-bit fixed point number
- float y = 3.14; // 32-bit floating point number

Control Structures

Python:

```
if x > 0:
print("Positive")
else:
print("Non-positive")
```

C:

```
if (x > 0) {
printf("Positive\n");
} else {
printf("Non-positive\n");
}
```

Loops

Python:

```
for i in range(5):
print(i)
```

C:

```
for (int i = 0; i < 5; i++) {
printf("%d\n", i);
}
```

Functions

Python:

- def add(a, b):
- return a + b

C:

- int add(int a, int b) {
- return a + b;
- }

Arrays vs Lists

Python:

- arr = [1, 2, 3, 4]
- print(arr[2])

C:

- int arr[4] = {1, 2, 3, 4};
- printf("%d\n", arr[2]);

Strings

Python:

- s = "Hello"
- print(len(s))

C:

- char s[] = "Hello";
- printf("%lu\n", strlen(s));

Memory Management

- Python
  - Automatic garbage collection
- C
  - `int *p = malloc(sizeof(int) * 5);`
  - `free(p);`

Pointers

- C Example:

```
int x = 10;
int *p = &x;
printf("%d\n", *p); // dereference
```

Structs vs Classes

Python:

- class Point:
- def __init__(self, x, y):
- self.x = x
- self.y = y

C:

- struct Point {
- int x;
- int y;
- };

Python ↔ C Interoperability

- Python
  - Extensively use C-based Python libraries
- Example
  - NumPy, TensorFlow use C under the hood.
- For Performance
  - Python Alone is TOO SLOW
  - Must build and use C/C++ Library

Why CUDA?

- CPUs
  - A Few Powerful cores
  - Good for sequential tasks.
- GPUs
  - Thousands of lightweight cores
  - Ideal for parallel tasks.
- CUDA
  - NVIDIA's platform to program GPUs using C/C++.
- Used in AI, simulations, graphics, data science.

CUDA vs C

- C: Programs run on CPU.
- CUDA C: C + extensions for GPU programming.
- Functions can run on CPU (host) or GPU (device).
- Explicit memory management between CPU and GPU.

CUDA Programming Model

- Host (CPU) and Device (GPU).
- Functions:

```
__host__ - runs on CPU
__device__ - runs on GPU
__global__ - GPU kernel callable from CPU
```

- Threads grouped into blocks and grids.

Hello Example

C Program:

- printf("Hello from CPU\n");

CUDA Program:

- #include <stdio.h>
- __global__ void hello() {
- printf("Hello from GPU!\n");
- }
- int main() {
- hello<<<1, 1>>>();
- cudaDeviceSynchronize();
- return 0;
- }

Example: Vector Addition

- CPU C Code:

```
for (int i = 0; i < N; i++) {
    C[i] = A[i] + B[i];
}
```

- CUDA Kernel:

```
__global__ void vecAdd(int *A, int *B, int *C,
int N) {
    int i = blockIdx.x * blockDim.x +
threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}
```

- Launch:

```
vecAdd<<<(N+255)/256, 256>>>(A, B, C, N);
```

Memory in CUDA

- Host Memory (CPU RAM) vs Device Memory (GPU VRAM).
- Explicit copies:

```
cudaMalloc((void**)&d_A, size);
cudaMemcpy(d_A, h_A,
size,cudaMemcpyHostToDevice);
```

- Types:
  - Global (slow, big)
  - Shared (fast, per block)
  - Registers (fastest, per thread)

Shared Memory Example

```
__global__ void addShared(int *A, int *B, int *C)
{
    __shared__ int temp[256];
    int tid = threadIdx.x;
    temp[tid] = A[tid] + B[tid];
    __syncthreads();
    C[tid] = temp[tid];
}
```

- Shared memory allows fast collaboration within a block.

Performance Considerations

- - Use many threads to hide latency.
- - Coalesced memory access = faster.
- - Minimize CPU↔GPU transfers.
- - Use shared memory wisely.
- - Profile with nvprof or nsight.

CUDA vs Multithreading in C

- C with pthreads/OpenMP
  - Parallel on CPU only.
- CUDA: thousands of GPU threads.
- GPU excels at data-parallel problems.

CUDA Applications

- Deep Learning (PyTorch, TensorFlow)
- Image processing
- Physics simulations
- Financial modeling

C to CUDA

- CUDA extends C for GPU programming.
- Kernels, threads, blocks, memory are key concepts.
- Workflow: Allocate → Copy to GPU → Launch kernel → Copy back.
- Learn memory hierarchy, profiling, optimization.

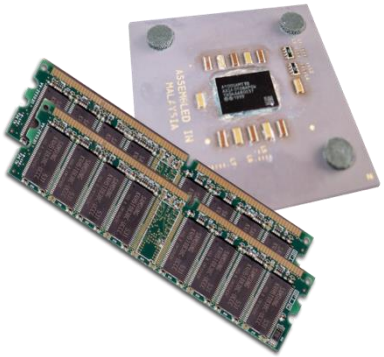
CUDA Internals

- CUDA Architecture
  - Expose GPU parallelism for general-purpose computing
  - Retain performance
- CUDA C/C++
  - Based on industry-standard C/C++
  - Small set of extensions to enable heterogeneous programming
  - Straightforward APIs to manage devices, memory etc.

© NVIDIA 2013

Heterogeneous Computing

- Terminology:
  - *Host* The CPU and its memory (host memory)
  - *Device* The GPU and its memory (device memory)



Host

Device

© NVIDIA 2013

Heterogeneous Computing

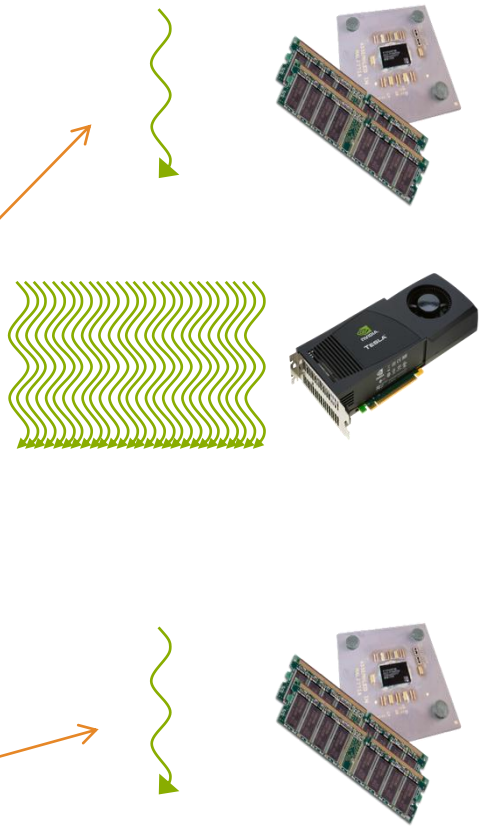
```
#include <iostream>
#include <algorithm>

using namespace std;

#define N        1024
#define RADIUS     3
#define BLOCK_SIZE 16

__global__ void stencil_1d(int *in, int *out) {
        __shared__ int temp[BLOCK_SIZE + 2 * RADIUS];
        int gindex = threadIdx.x + blockIdx.x * blockDim.x;
        int lindex = threadIdx.x + RADIUS;

        // Read input elements into shared memory
        temp[lindex] = in[gindex];
        if (threadIdx.x < RADIUS) {
                temp[lindex - RADIUS] = in[gindex - RADIUS];
                temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];
        }

        // Synchronize (ensure all the data is available)
        __syncthreads();

        // Apply the stencil
        int result = 0;
        for (int offset = -RADIUS ; offset <= RADIUS ; offset++)
                result += temp[lindex + offset];

        // Store the result
        out[gindex] = result;
}

void fill_ints(int *x, int n) {
        fill_n(x, n, 1);
}

int main(void) {
        int *in, *out;          // host copies of a, b, c
        int *d_in, *d_out;      // device copies of a, b, c
        int size = (N + 2*RADIUS) * sizeof(int);

        // Alloc space for host copies and setup values
        in  = (int *)malloc(size); fill_ints(in,  N + 2*RADIUS);
        out = (int *)malloc(size); fill_ints(out, N + 2*RADIUS);

        // Alloc space for device copies
        cudaMalloc((void **)&d_in,  size);
        cudaMalloc((void **)&d_out, size);

        // Copy to device
        cudaMemcpy(d_in,  in,  size, cudaMemcpyHostToDevice);
        cudaMemcpy(d_out, out, size, cudaMemcpyHostToDevice);

        // Launch stencil_1d() kernel on GPU
        stencil_1d<<<N/BLOCK_SIZE,BLOCK_SIZE>>>(d_in + RADIUS,
d_out + RADIUS);

        // Copy result back to host
        cudaMemcpy(out, d_out, size, cudaMemcpyDeviceToHost);

        // Clean up
        free(in); free(out);
        cudaFree(d_in); cudaFree(d_out);
        return 0;
}
```

parallel fn

serial code

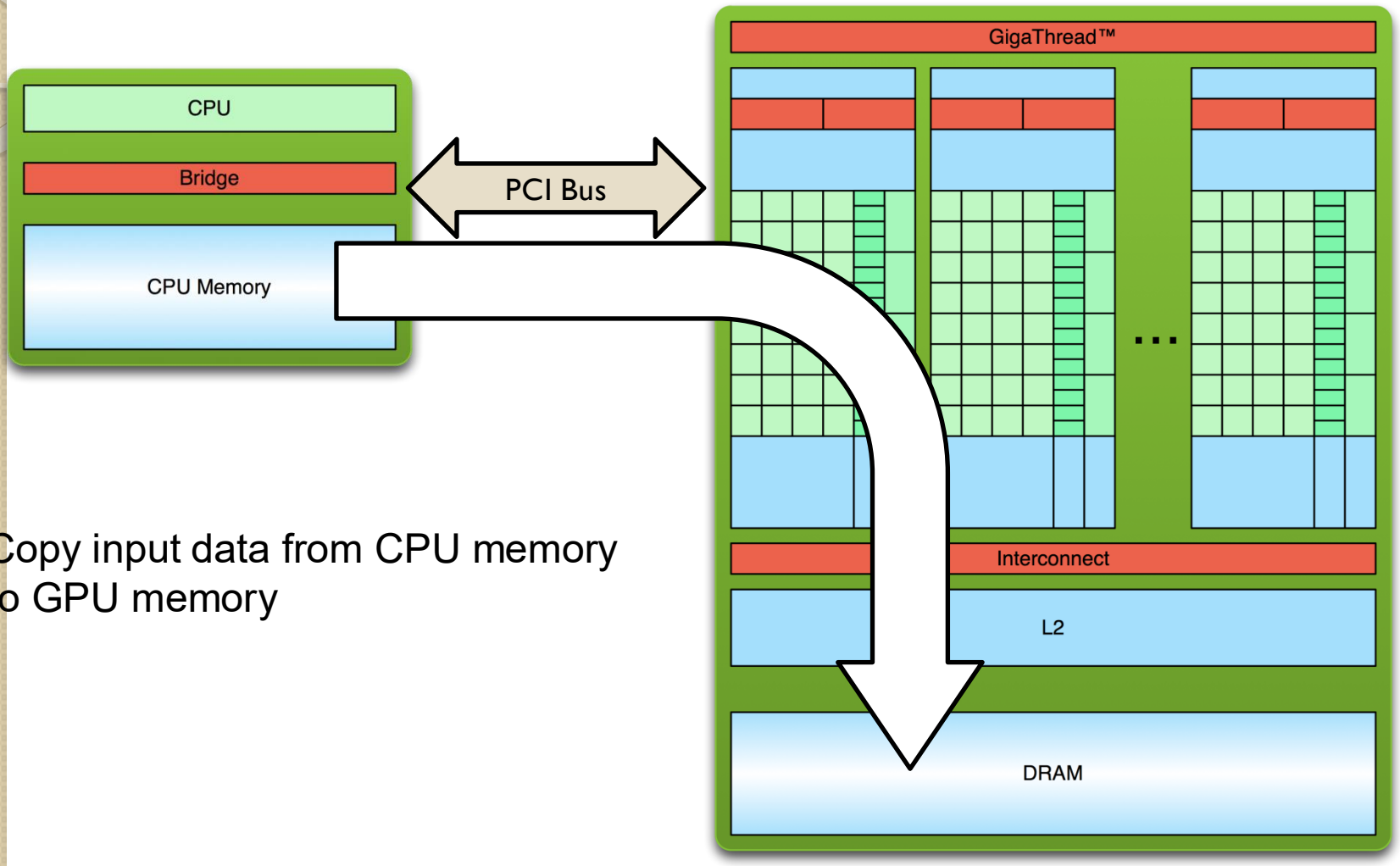
parallel code

serial code



© NVIDIA 2013

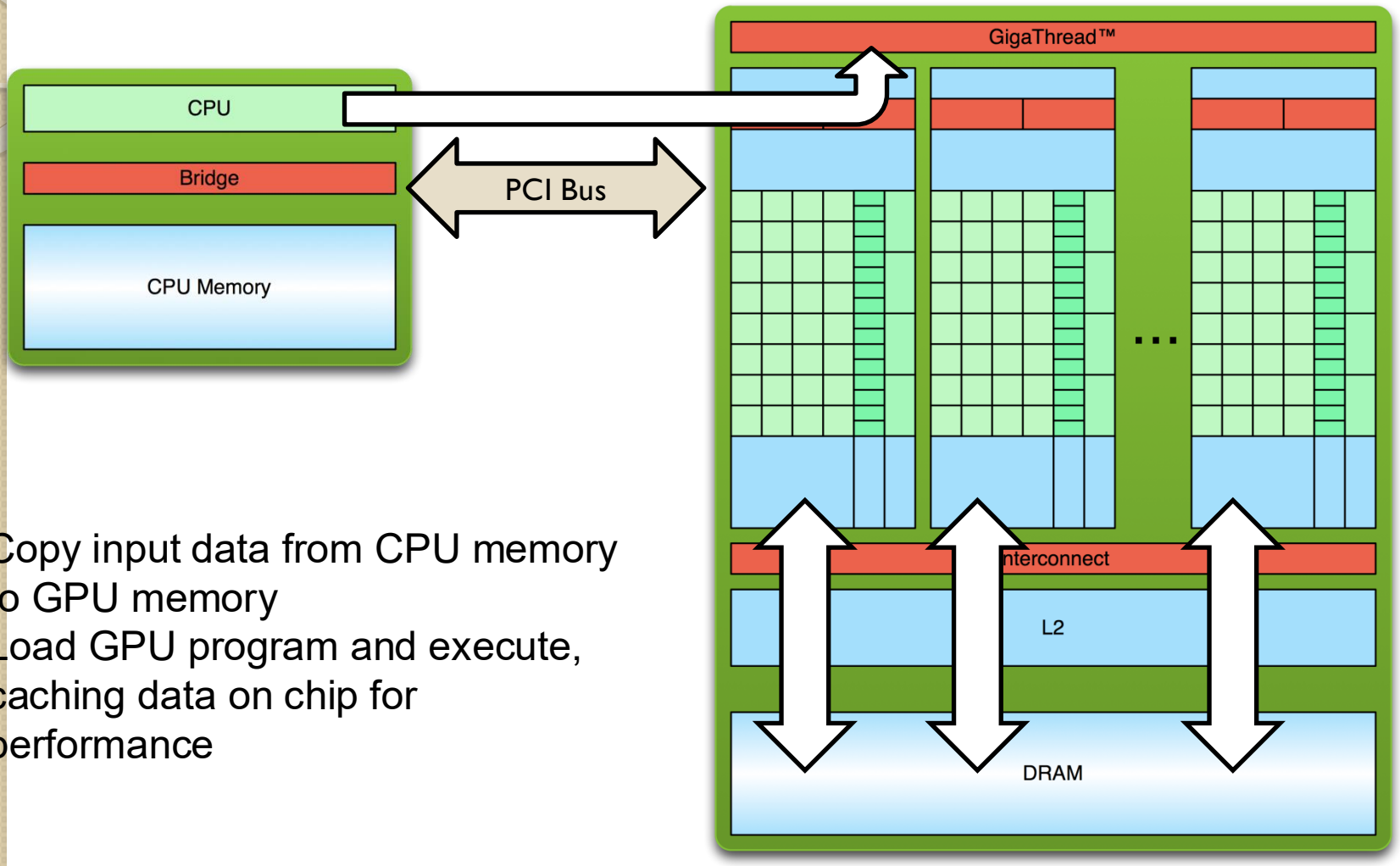
Simple Processing Flow



1. Copy input data from CPU memory to GPU memory

© NVIDIA 2013

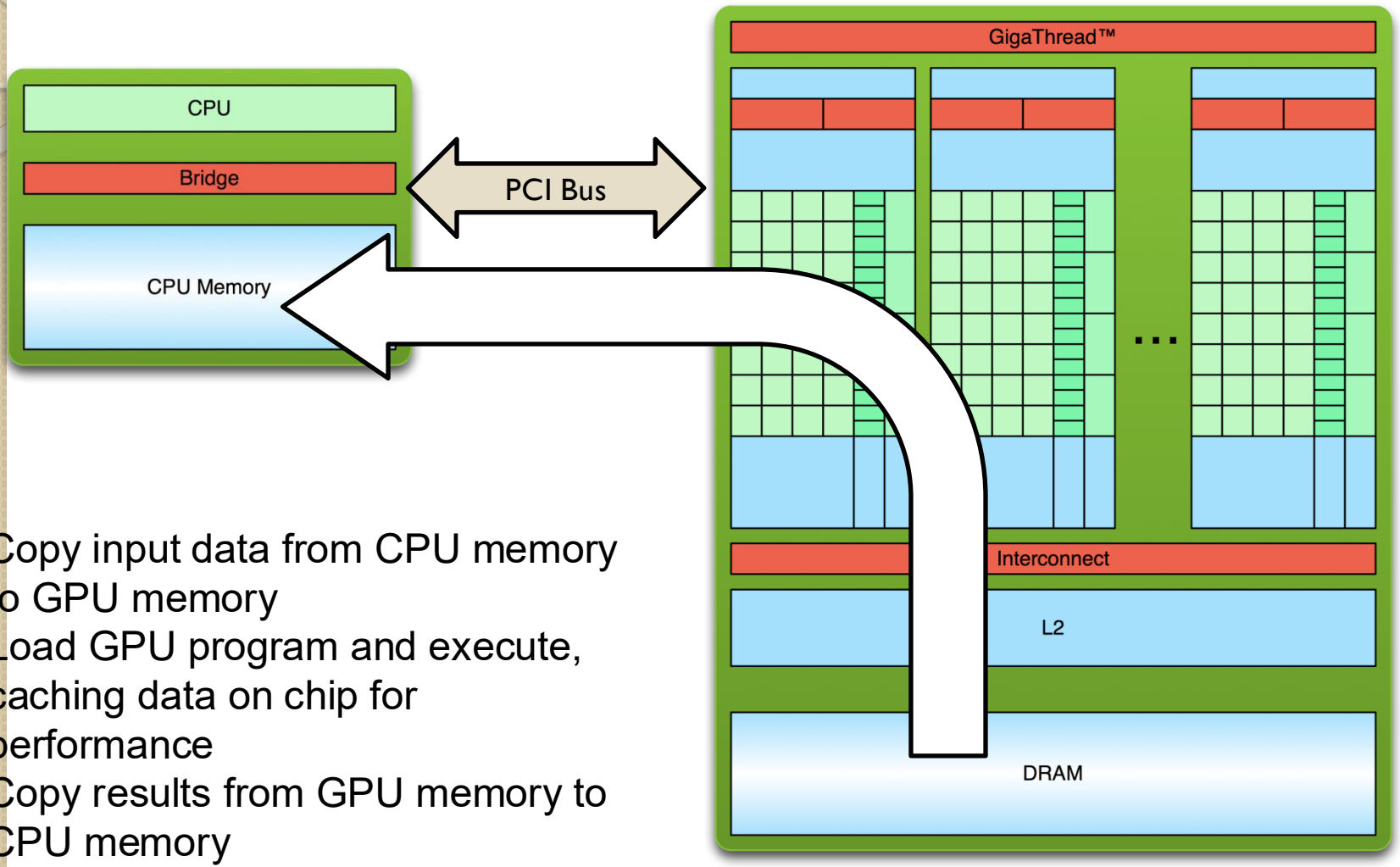
Simple Processing Flow



1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute, caching data on chip for performance

© NVIDIA 2013

Simple Processing Flow



1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute, caching data on chip for performance
3. Copy results from GPU memory to CPU memory

© NVIDIA 2013

Hello World!

```
int main(void) {
        printf("Hello World!\n");
        return 0;
}
```

- Standard C that runs on the host
- NVIDIA compiler (nvcc) can be used to compile programs with no *device* code
- Output:

```
$ nvcc hello_world.cu
$ a.out
Hello World!
$
```

© NVIDIA 2013

Hello World! with Device Code

```
__global__ void mykernel(void) {
}

int main(void) {
        mykernel<<<1,1>>>();
        printf("Hello World!\n");
        return 0;
}
```

- Two new syntactic elements…

© NVIDIA 2013

Hello World! with Device Code

```
__global__ void mykernel(void) {
}
```

- CUDA C/C++ keyword `__global__` indicates a function that:
  - Runs on the device
  - Is called from host code

- `nvcc` separates source code into host and device components
  - Device functions (e.g. `mykernel()`) processed by NVIDIA compiler
  - Host functions (e.g. `main()`) processed by standard host compiler
    - `gcc, cl.exe`

© NVIDIA 2013

Hello World! with Device Code

```
mykernel<<<1,1>>>();
```

- Triple angle brackets mark a call from *host* code to *device* code
  - Also called a "kernel launch"
  - We'll return to the parameters (1,1) in a moment
- That's all that is required to execute a function on the GPU!

© NVIDIA 2013

Hello World! with Device Code

```
__global__ void mykernel(void){
}
int main(void) {
        mykernel<<<1,1>>>();
        printf("Hello World!\n");
        return 0;
}
```

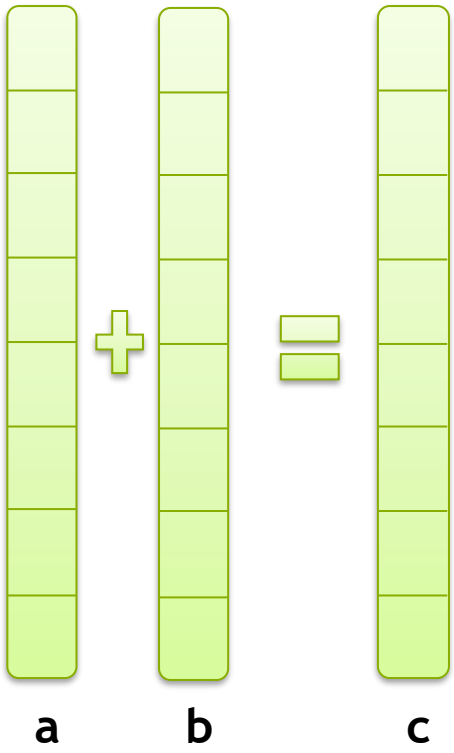
- `mykernel()` does nothing
- Output:

```
$ nvcc hello.cu
$ a.out
Hello World!
```

© NVIDIA 2013

Parallel Programming in CUDA C/C++

- But wait… GPU computing is about massive parallelism!
- We need a more interesting example…
- We'll start by adding two integers and build up to vector addition



© NVIDIA 2013

Addition on the Device

- A simple kernel to add two integers

```
__global__ void add(int *a, int *b, int *c)
{
        *c = *a + *b;
}
```

- As before `__global__` is a CUDA C/C++ keyword meaning
  - `add()` will execute on the device
  - `add()` will be called from the host

© NVIDIA 2013

Addition on the Device

- Note that we use pointers for the variables

```
__global__ void add(int *a, int *b, int *c) {
        *c = *a + *b;
}
```

- `add()` runs on the device, so `a`, `b` and `c` must point to device memory

- We need to allocate memory on the GPU

© NVIDIA 2013

Memory Management

- Host and device memory are separate entities
  - *Device* pointers point to GPU memory
    - May be passed to/from host code
    - May *not* be dereferenced in host code
  - *Host* pointers point to CPU memory
    - May be passed to/from device code
    - May *not* be dereferenced in device code

- Simple CUDA API for handling device memory
  - `cudaMalloc()`, `cudaFree()`, `cudaMemcpy()`
  - Similar to the C equivalents `malloc()`, `free()`, `memcpy()`

© NVIDIA 2013

Addition on the Device: `add()`

- Returning to our `add()` kernel

```
__global__ void add(int *a, int *b, int *c) {
        *c = *a + *b;
}
```

- Let's take a look at main()…

© NVIDIA 2013

Addition on the Device: `main()`

```
int main(void) {
        int a, b, c;                    // host copies of a, b, c
        int *d_a, *d_b, *d_c;           // device copies of a, b, c
        int size = sizeof(int);

        // Allocate space for device copies of a, b, c
        cudaMalloc((void **)&d_a, size);
        cudaMalloc((void **)&d_b, size);
        cudaMalloc((void **)&d_c, size);

        // Setup input values
        a = 2;
        b = 7;
```

© NVIDIA 2013

Addition on the Device: `main()`

```
// Copy inputs to device
cudaMemcpy(d_a, &a, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_b, &b, size, cudaMemcpyHostToDevice);

// Launch add() kernel on GPU
add<<<1,1>>>(d_a, d_b, d_c);

// Copy result back to host
cudaMemcpy(&c, d_c, size, cudaMemcpyDeviceToHost);

// Cleanup
cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
return 0;
```

© NVIDIA 2013

Moving to Parallel

- GPU computing is about massive parallelism
  - So how do we run code in parallel on the device?

```
add<<< 1, 1 >>>();

add<<< N, 1 >>>();
```

- Instead of executing `add()` once, execute N times in parallel

© NVIDIA 2013

Vector Addition on the Device

- With `add()` running in parallel we can do vector addition
- Terminology: each parallel invocation of `add()` is referred to as a block
  - The set of blocks is referred to as a grid
  - Each invocation can refer to its block index using `blockIdx.x`

```
__global__ void add(int *a, int *b, int *c) {
                c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];
        }
```

- By using `blockIdx.x` to index into the array, each block handles a different index

© NVIDIA 2013

Vector Addition on the Device

```
__global__ void add(int *a, int *b, int *c) {
        c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];
}
```

- On the device, each block can execute in parallel:

Block 0	Block 1	Block 2	Block 3
`c[0] = a[0] + b[0];`	`c[1] = a[1] + b[1];`	`c[2] = a[2] + b[2];`	`c[3] = a[3] + b[3];`

© NVIDIA 2013

Vector Addition on the Device: `add()`

- Returning to our parallelized `add()` kernel

```
__global__ void add(int *a, int *b, int *c) {
        c[blockIdx.x] = a[blockIdx.x] +
b[blockIdx.x];
}
```

- Let's take a look at main()…

© NVIDIA 2013

Vector Addition on the Device:

main()

```
#define N 512
int main(void) {
    int *a  *b  *c                    // host copies of a, b, c
    int *d_a, *d_b, *d_c;  // device copies of a, b, c
    int size = N * sizeof(int);

    // Alloc space for device copies of a, b, c
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    // Alloc space for host copies of a, b, c and setup
input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);
```

© NVIDIA 2013

Vector Addition on the Device: main()

```
// Copy inputs to device
cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

// Launch add() kernel on GPU with N blocks
add<<<N,1>>>(d_a, d_b, d_c);

// Copy result back to host
cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

// Cleanup
free(a); free(b); free(c);
cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
return 0;
```

© NVIDIA 2013

CUDA Threads

- Terminology: a block can be split into parallel threads

- Let's change `add()` to use parallel *threads* instead of parallel *blocks*

```
__global__ void add(int *a, int *b, int *c) {
    c[threadIdx.x] = a[threadIdx.x] + b[threadIdx.x];
}
```

- We use `threadIdx.x` instead of `blockIdx.x`

- Need to make one change in `main()`...

© NVIDIA 2013

Vector Addition Using Threads: main()

```
#define N 512
int main(void) {
    int *a, *b, *c;                    // host copies of a, b, c
    int *d_a, *d_b, *d_c;              // device copies of a, b, c
    int size = N * sizeof(int);

    // Alloc space for device copies of a, b, c
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    // Alloc space for host copies of a, b, c and setup input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);
```

© NVIDIA 2013

Vector Addition Using Threads: main()

```
 // Copy inputs to device
cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

// Launch add() kernel on GPU with N threads
add<<<1,N>>>(d_a, d_b, d_c);

// Copy result back to host
cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

// Cleanup
free(a); free(b); free(c);
cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
return 0;
```

© NVIDIA 2013

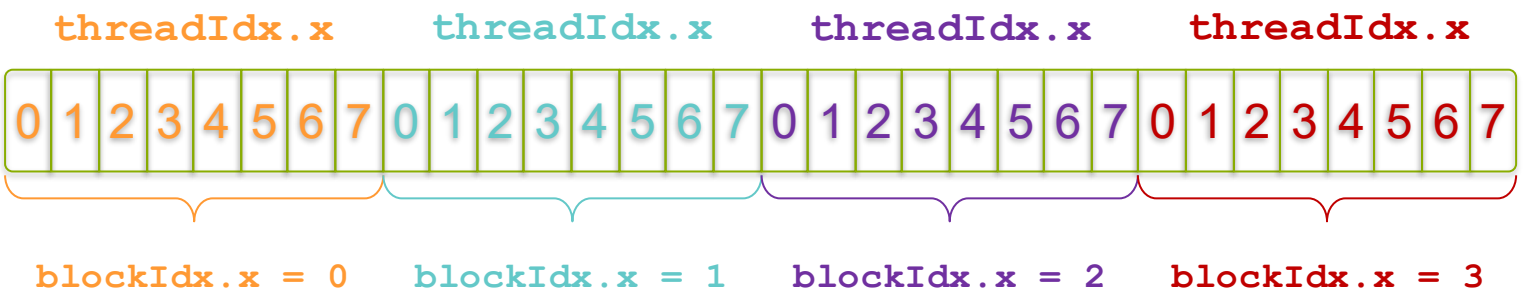
Combining Blocks and Threads

- We've seen parallel vector addition using:
  - Many blocks with one thread each
  - One block with many threads
- Let's adapt vector addition to use both blocks and threads
- Why? We'll come to that…
- First let's discuss data indexing…

© NVIDIA 2013

Indexing Arrays with Blocks and Threads

- No longer as simple as using `blockIdx.x` and `threadIdx.x`
  - Consider indexing an array with one element per thread (8 threads/block)



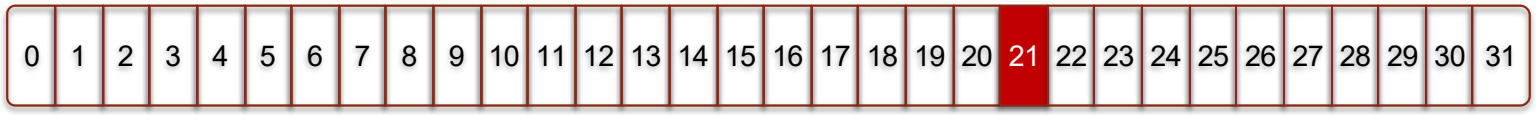
- With M threads/block a unique index for each thread is given by:

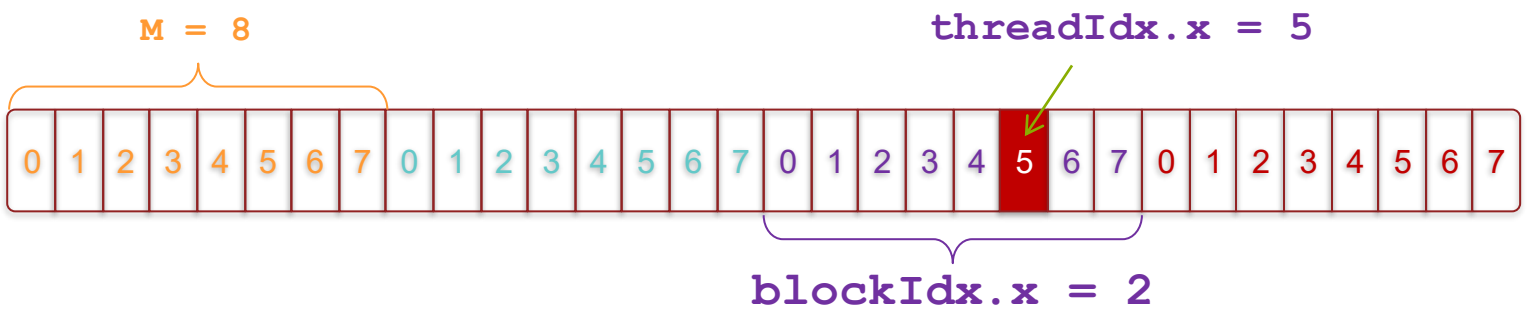
```
int index = threadIdx.x + blockIdx.x * M;
```

© NVIDIA 2013

Indexing Arrays: Example

- Which thread will operate on the red element?






```
int index = threadIdx.x + blockIdx.x * M;
          =       5     +     2      * 8;
          = 21;
```

© NVIDIA 2013

Vector Addition with Blocks and Threads

- Use the built-in variable `blockDim.x` for threads per block

```
int index = threadIdx.x + blockIdx.x * blockDim.x;
```

- Combined version of `add()` to use parallel threads *and* parallel blocks

```
__global__ void add(int *a, int *b, int *c) {
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    c[index] = a[index] + b[index];
}
```

- What changes need to be made in `main()`?

© NVIDIA 2013

Addition with Blocks and Threads: `main()`

```
#define N (2048*2048)
#define THREADS_PER_BLOCK 512
int main(void) {
    int *a, *b, *c;                     // host copies of a, b, c
    int *d_a, *d_b, *d_c;               // device copies of a, b, c
    int size = N * sizeof(int);

    // Alloc space for device copies of a, b, c
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    // Alloc space for host copies of a, b, c and setup input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);
```

© NVIDIA 2013

Addition with Blocks and Threads: `main()`

```
    // Copy inputs to device
    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

    // Launch add() kernel on GPU
    add<<<N/THREADS_PER_BLOCK THREADS_PER_BLOCK>>>(d_a, d_b, d_c);

    // Copy result back to host
    cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

    // Cleanup
    free(a); free(b); free(c);
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    return 0;
}
```

© NVIDIA 2013

Handling Arbitrary Vector Sizes

- Typical problems are not friendly multiples of `blockDim.x`

- Avoid accessing beyond the end of the arrays:

```
__global__ void add(int *a, int *b, int *c, int n) {
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    if (index < n)
        c[index] = a[index] + b[index];
}
```

- Update the kernel launch:

```
add<<<(N + M-1) / M,M>>>(d_a, d_b, d_c, N);
```

© NVIDIA 2013

Why Bother with Threads?

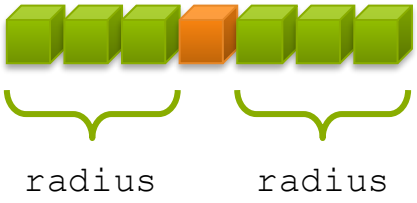
- Threads seem unnecessary
  - They add a level of complexity
  - What do we gain?
- Unlike parallel blocks, threads have mechanisms to:
  - Communicate
  - Synchronize
- To look closer, we need a new example…

© NVIDIA 2013

1D Stencil

- Consider applying a 1D stencil to a 1D array of elements
  - Each output element is the sum of input elements within a radius

- If radius is 3, then each output element is the sum of 7 input elements:

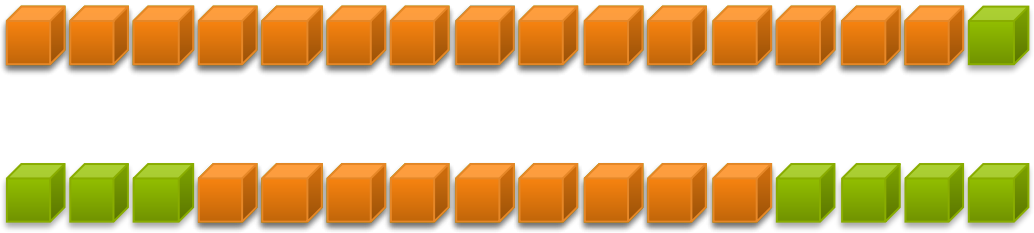


© NVIDIA 2013

Implementing Within a Block

- Each thread processes one output element
  - blockDim.x elements per block

- Input elements are read several times
  - With radius 3, each input element is read seven times



© NVIDIA 2013

Sharing Data Between Threads

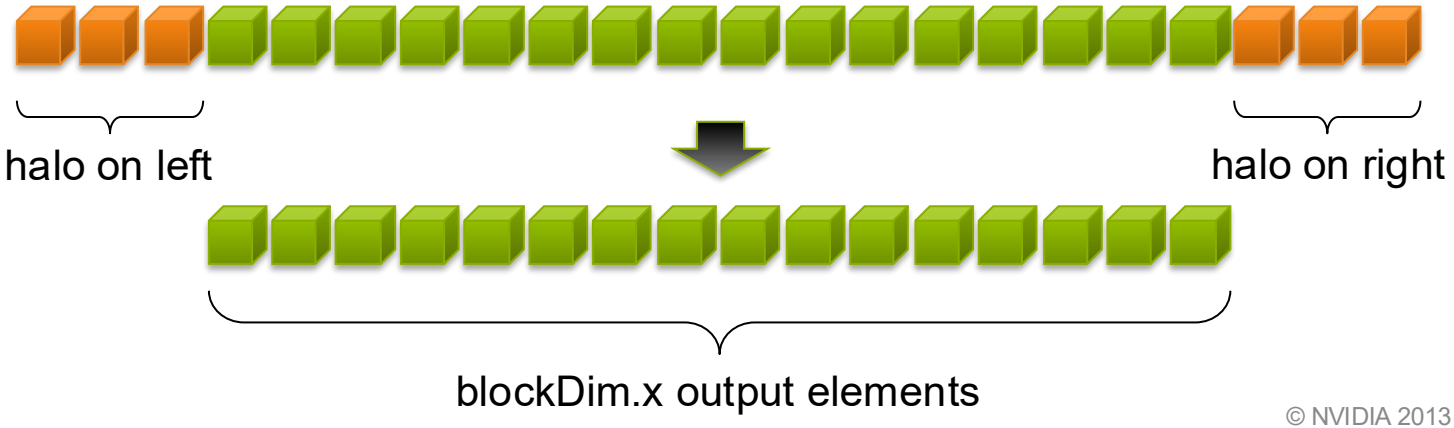
- Terminology: within a block, threads share data via shared memory
- Extremely fast on-chip memory, user-managed
- Declare using `__shared__`, allocated per block
- Data is not visible to threads in other blocks

© NVIDIA 2013

Implementing With Shared Memory

- Cache data in shared memory
  - Read (blockDim.x + 2 * radius) input elements from global memory to shared memory
  - Compute blockDim.x output elements
  - Write blockDim.x output elements to global memory

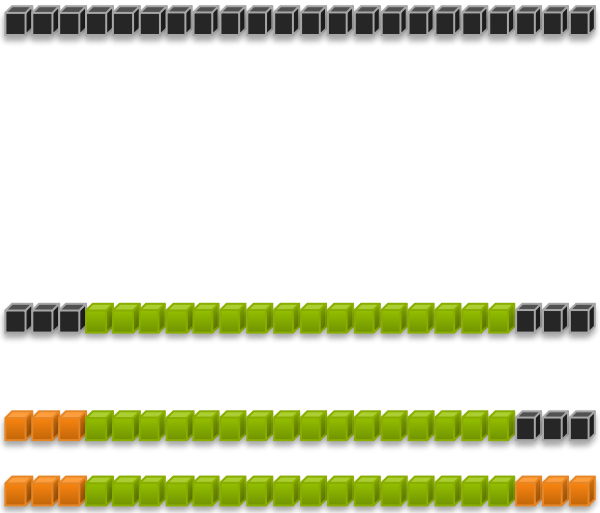
  - Each block needs a halo of radius elements at each boundary



© NVIDIA 2013

Stencil Kernel

```
__global__ void stencil_1d(int *in, int *out) {
  __shared__ int temp[BLOCK_SIZE + 2 * RADIUS];
  int gindex = threadIdx.x + blockIdx.x * blockDim.x;
  int lindex = threadIdx.x + RADIUS;

  // Read input elements into shared memory
  temp[lindex] = in[gindex];
  if (threadIdx.x < RADIUS) {
    temp[lindex - RADIUS] = in[gindex - RADIUS];
    temp[lindex + BLOCK_SIZE] =
      in[gindex + BLOCK_SIZE];
  }
```



© NVIDIA 2013

Stencil Kernel

```
  // Apply the stencil
  int result = 0;
  for (int offset = -RADIUS ; offset <= RADIUS ; offset++)
    result += temp[lindex + offset];

  // Store the result
  out[gindex] = result;
}
```

© NVIDIA 2013

Data Race!

- The stencil example will not work…
- Suppose thread 15 reads the halo before thread 0 has fetched it…

```
temp[lindex] = in[gindex];          Store at temp[18]
if (threadIdx.x < RADIUS) {
  temp[lindex - RADIUS = in[gindex - RADIUS];     Skipped, threadIdx > RADIUS
  temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];
}

int result = 0;
result += temp[lindex + 1];         Load from temp[19]
```

© NVIDIA 2013

__syncthreads()

- `void __syncthreads();`

- Synchronizes all threads within a block
  - Used to prevent RAW / WAR / WAW hazards

- All threads must reach the barrier
  - In conditional code, the condition must be uniform across the block

© NVIDIA 2013

Stencil Kernel

```
__global__ void stencil_1d(int *in, int *out) {
    __shared__ int temp[BLOCK_SIZE + 2 * RADIUS];
    int gindex = threadIdx.x + blockIdx.x * blockDim.x;
    int lindex = threadIdx.x + radius;

    // Read input elements into shared memory
    temp[lindex] = in[gindex];
    if (threadIdx.x < RADIUS) {
        temp[lindex - RADIUS] = in[gindex - RADIUS];
        temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];
    }

    // Synchronize (ensure all the data is available)
    __syncthreads();
```

© NVIDIA 2013

Stencil Kernel

```
    // Apply the stencil
    int result = 0;
    for (int offset = -RADIUS ; offset <= RADIUS ; offset++)
        result += temp[lindex + offset];

    // Store the result
    out[gindex] = result;
}
```

© NVIDIA 2013

Coordinating Host & Device

- Kernel launches are asynchronous
  - Control returns to the CPU immediately

- CPU needs to synchronize before consuming the results

`cudaMemcpy()`	Blocks the CPU until the copy is complete Copy begins when all preceding CUDA calls have completed
`cudaMemcpyAsync()`	Asynchronous, does not block the CPU
`cudaDeviceSynchronize()`	Blocks the CPU until all preceding CUDA calls have completed

© NVIDIA 2013

Reporting Errors

- All CUDA API calls return an error code (`cudaError_t`)
  - Error in the API call itself

    OR
  - Error in an earlier asynchronous operation (e.g. kernel)

- Get the error code for the last error:

```
cudaError_t cudaGetLastError(void)
```

- Get a string to describe the error:

```
char *cudaGetErrorString(cudaError_t)
```

```
printf("%s\n", cudaGetErrorString(cudaGetLastError()));
```

© NVIDIA 2013

Device Management

- Application can query and select GPUs

```
cudaGetDeviceCount(int *count)
cudaSetDevice(int device)
cudaGetDevice(int *device)
cudaGetDeviceProperties(cudaDeviceProp *prop, int device)
```

- Multiple threads can share a device

- A single thread can manage multiple devices

  `cudaSetDevice(i)` to select current device

  `cudaMemcpy(…)` for peer-to-peer copies†

† requires OS and device support

© NVIDIA 2013

Introduction to CUDA C/C++

- What have we learned?
  - Write and launch CUDA C/C++ kernels
    - `__global__, blockIdx.x, threadIdx.x, <<<>>>`
  - Manage GPU memory
    - `cudaMalloc(), cudaMemcpy(), cudaFree()`
  - Manage communication and synchronization
    - `__shared__, __syncthreads()`
    - `cudaMemcpy()` VS `cudaMemcpyAsync(), cudaDeviceSynchronize()`

© NVIDIA 2013

Compute Capability

- The **compute capability** of a device describes its architecture, e.g.
  - Number of registers
  - Sizes of memories
  - Features & capabilities

Compute Capability	Selected Features (see CUDA C Programming Guide for complete list)	Tesla models
1.0	Fundamental CUDA support	870
1.3	Double precision, improved memory accesses, atomics	10-series
2.0	Caches, fused multiply-add, 3D grids, surfaces, ECC, P2P, concurrent kernels/copies, function pointers, recursion	20-series

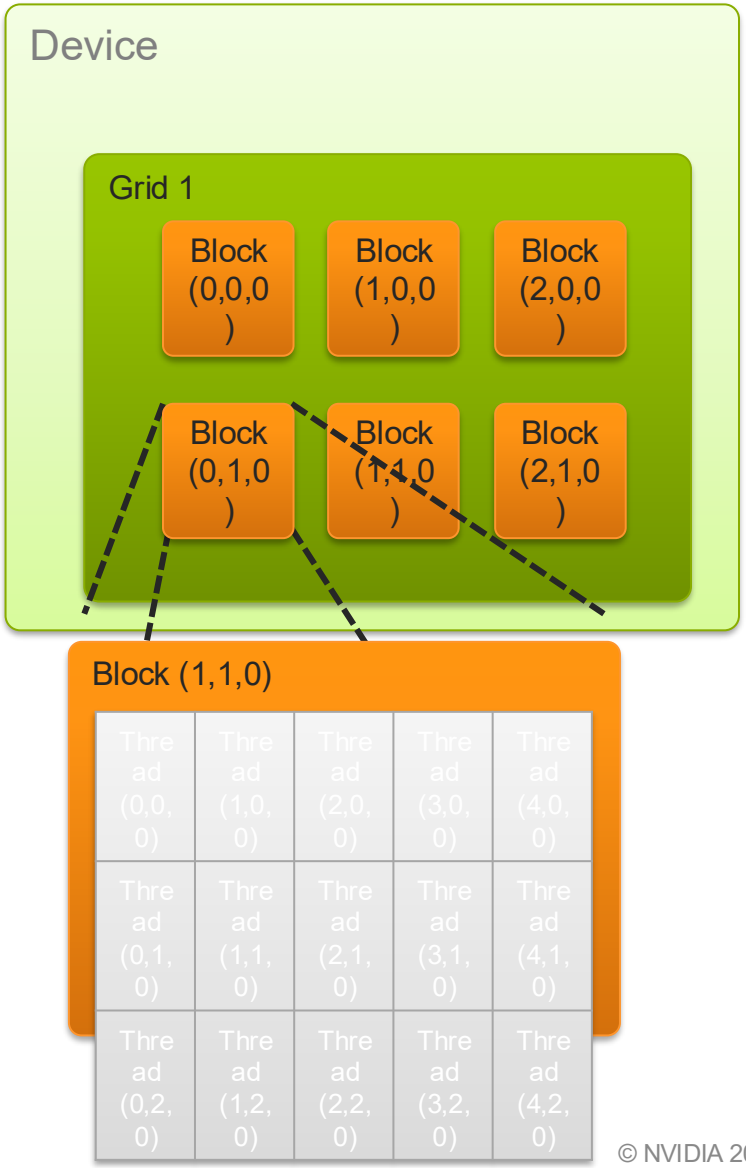
- The following presentations concentrate on Fermi devices
  - Compute Capability >= 2.0

© NVIDIA 2013

IDs and Dimensions

- A kernel is launched as a grid of blocks of threads
  - `blockIdx` and `threadIdx` are 3D
  - We showed only one dimension (`x`)

- Built-in variables:
  - `threadIdx`
  - `blockIdx`
  - `blockDim`
  - `gridDim`



© NVIDIA 2013